MREN 103 – Mechatronics and Robotics Design I
Mechatronics and Robotics Engineering Program
Queen's University, Kingston

Group #30
Laboratory #2 – Debugging and the Servomotor

Student Names: **Brady Cabral and Emily Vargas**
Instructor Name: Prof. Surgenor
Date Report Submitted: February 25, 2026

Summary:
A prewired breadboard was used as an introduction to servomotors and code debugging. Activities included: a) wiring the servomotor to the breadboard, b) debugging a supplied program to change its sequence of operations and c) recalibrating the servomotor to ensure its smooth operation. The objective of this lab is to learn about the hardware and software associated with the operation of a servomotor, as well as to gain further confidence with the Arduino language by debugging a program written by someone else.

Hardware:
The setup for the breadboard used in the lab is pictured in **Fig. 1**. Components on the breadboard include: ItsyBitsy microcontroller, resistors, LEDs, pushbutton, assorted jumper wires, and a servomotor.

Software:
The program 'Group30Lab2GoodServo' shown in **Appendix A** is a demo which cycles the servo through multiple angles and reset when the red button is pressed. The program first initializes all variables and sets the functions of the pins, it then flashes the green LED until the red button shown in **Fig. 1.** is pressed. The yellow LED is turned on, and the servo then moves from 90 degrees angle to 180 degrees then to 0 degrees and finally back to 90 degrees. The program then makes the green LED flash until the red button is pressed then after which the red LED flashes on for a second and the program loops back to moving the servo motor.

Results*:*
The diagnostic table developed in response to the program's errors introduced by the instructor is given as **Table 1**. The most problematic error was fixing the motor stalling at 180 degrees. Initially, the servomotor failed to turn at all. Although it was initially thought to be due to a bug in the code, it was later discovered to be due to a wiring error introduced from the initial wiring step. However, once the jumper was swapped to the correct position, the motor was able to rotate. After fixing this hardware issue, identifying the incorrect arithmetic operator was fairly straightforward.

Conclusions:
In this lab, we strengthened our ability to quickly identify and correct issues, both in hardware and software. We also gained some knowledge on the pulse width required to make the servomotor turn effectively during the recalibration process. Overall, the brief lecture in the beginning of class was also a helpful introduction to how a servomotor differs from a regular motor.

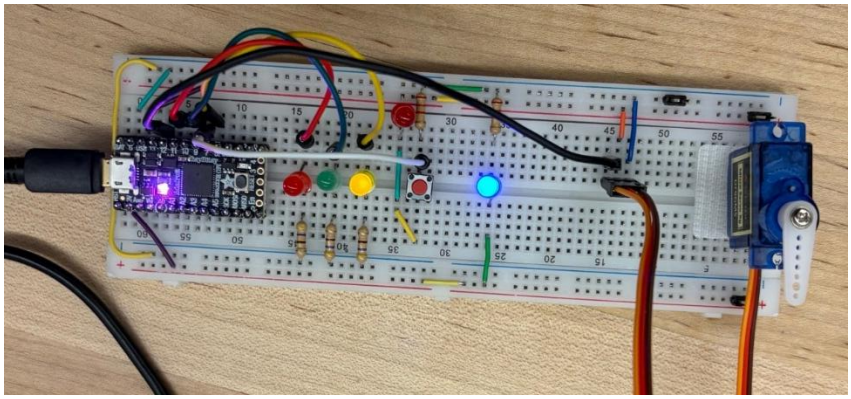

Fig. 1. Breadboard setup for Lab #2 with microcontroller, LEDs, resistors and a servo motor shown.

Table 1. Diagnostic table for wiring errors in **Lab #1**

Number	Symptom	Source of Problem	Fix
1	Won't compile, 'BUTTON' was not declared in this scope	BUTTON variable was declared lowercase.	Change 'button' to 'BUTTON'
2	error: expected declaration before '}' token	Extra end bracket for loop function	Remove extra end bracket
3	error: 'myAngle' was not declared in this scope	No declaration of 'myAngle'	Declare a global variable for 'myAngle'
4	Green LED won't turn on	Green pin not defined to be output	Set stat for green pin
5	Servo motor stalls at 180 degrees	Incorrect arithmetic operator used for 'myAngle'	Changed '++' to '--'
6	Program doesn't wait for button press to restart	No code that tells the ItsyBitsy to wait for a button press	Added a while loop, that flashes the green LED, with the condition that button must be pressed. Also turned off Yellow LED when Green LED is flashing.

Appendix A - Program Listing:

```
/*****************************************************
  Group30Lab2GoodServo
  B. Cabral, E. Vargas, 25/02/2026

  - Bugs fixed
  - when working, servo sweeps to 180, then 0, and
stops at 90 deg
  - yellow LED on when running, green LED flashing
when ready
  - recommend start with servo at 90 deg (point straight
ahead)

*****************************************************/

// Pin Assignments
int RED = 10;          //red LED Pin
int GRN = 9;          //yellow LED Pin
int YLW = 5;          //green LED Pin
int BUTTON = 7;        //pushbutton
int servoPin = 12;    //servo connected to digital pin 12
int pulseWidth;       //servoPulse function variable

int myAngle = 0;      // Desired angle of servo

// Set-up Routine
void setup() {

// set stat of pins
  pinMode(YLW, OUTPUT);
  pinMode(RED, OUTPUT);
  pinMode(BUTTON, INPUT);
  pinMode(servoPin, OUTPUT);
  pinMode(GRN, OUTPUT);

// set up serial
   Serial.begin(9600);
   Serial.println("Press button to start.");

// wait for button press
do{
      digitalWrite(GRN, HIGH);
      delay(125);
      digitalWrite(GRN, LOW);
      delay(125);
   }while(digitalRead(BUTTON) == LOW);
}

// Main Routine
void loop(){
    delay(1000);
    Serial.println("Servo sweeping.");
    turnOnLED(YLW);     // yellow indicates running

 // first sweep, starts at 90 deg and rotates to 180 deg
    for (myAngle=90; myAngle<=180; myAngle++)
    {
     servoPulse(servoPin, myAngle);
     delay(20);
     Serial.println(myAngle);
    }
    delay(500);

// second sweep, starts at 180 deg and rotates to 0 deg
    for (myAngle=180; myAngle>=0; myAngle--)
    {
     servoPulse(servoPin, myAngle);
     delay(20);
     Serial.println(myAngle);
    }
     delay(500);

// third sweep, starts at 0 deg and rotates to 90 deg
    for (myAngle=0; myAngle<=90; myAngle++)
    {
     servoPulse(servoPin, myAngle);
     delay(20);
     Serial.println(myAngle);
    }
    // Sets yellow LED to be off
    digitalWrite(YLW, LOW);
    // Waits for button press
    do{
       digitalWrite(GRN, HIGH);
       delay(125);
       digitalWrite(GRN, LOW);
       delay(125);
    }while(digitalRead(BUTTON) == LOW);

    turnOnLED(RED);        // red indicates end of a cycle
    delay(1000);           // pause before repeating cycle

    digitalWrite(RED,LOW);
    Serial.println(" ");   // generate a line feed
}


//*************** FUNCTIONS (subroutines) *************

void servoPulse(int servoPin, int myAngle)
{
```

```
  pulseWidth = map(myAngle,0,180,500,2500);  //
default values 550 and 2200
  digitalWrite(servoPin, HIGH);  //set servo high
  delayMicroseconds(pulseWidth);  //microsecond
pause
  digitalWrite(servoPin, LOW);  //set servo low
}

void turnOnLED(int COLOUR)
{
  digitalWrite(GRN, LOW);
  digitalWrite(YLW, LOW);
  digitalWrite(RED, LOW);
  digitalWrite(COLOUR, HIGH);
}
```